LAB MANUAL

Topic: Basics of Large Language Models (LLMs)
Subject: Artificial Intelligence / Natural Language Processing

Objective:

The objective of this lab manual is to help students understand the basics of Large Language Models (LLMs). After completing this lab, students will be able to explain what an LLM is, understand how the field moved from traditional NLP techniques to LLMs, describe (conceptually) how an LLM processes and generates text, identify popular LLMs and their common use cases, understand key LLM terminology, and get hands-on with tokenization, text generation, prompting, and embeddings using standard Python libraries. (Note: This lab focuses on concepts and hands-on usage; internal mathematical details and building a model from scratch are intentionally kept out of scope to save time.)

Part 1: Introduction to Large Language Models (LLMs)

1.1 What is a Large Language Model?

A **Large Language Model (LLM)** is an AI model trained on massive amounts of text data to understand and generate human-like language. Given some input text (a **prompt**), an LLM predicts the most likely next word (technically, the next **token**), and repeats this process to generate whole sentences, paragraphs, code, or conversations. Examples of LLMs include the GPT family, Claude, Gemini, LLaMA, and Mistral.

1.2 From Traditional NLP to LLMs

Traditional Natural Language Processing (NLP) relied on rule-based systems and smaller statistical/ML models (e.g., Naive Bayes, HMMs, small RNNs) that were usually trained for one narrow task at a time (e.g., just sentiment analysis, or just spam detection), often needing hand-crafted features. LLMs changed this by training one very large model on huge, general text corpora, so a single model can handle many different language tasks — translation, summarization, question answering, coding, and conversation — without being retrained from scratch for each one.

Aspect (Traditional NLP)	Description
Scope	Usually built for one specific task (e.g., only classification or only translation).
Features	Often required manually engineered linguistic features (POS tags, n-grams, etc.).
Data Needed	Could work with comparatively small, task-specific labeled datasets.
Generalization	Poor at handling tasks or phrasing it wasn't specifically trained for.

Example	A Naive Bayes model trained only to classify emails as spam or not spam.

Aspect (LLMs)	Description
Scope	One general-purpose model handles many language tasks via natural-language prompts.
Features	Learns its own internal representations directly from raw text; no manual feature design.
Data Needed	Pre-trained on massive, general text datasets (books, websites, code, etc.).
Generalization	Can often handle new, unseen tasks reasonably well just from a prompt (few-shot/zero-shot).
Example	A single LLM that can summarize, translate, and answer questions, just by changing the prompt.

1.3 How LLMs Work (Conceptually)

Under the hood, an LLM is built on an architecture called the **Transformer**. Without going into the internal mathematics, the overall flow of turning your prompt into a response looks like this:

1. **Tokenization:** The input text is broken into small pieces called **tokens** (roughly words or sub-words).
2. **Embedding:** Each token is converted into a list of numbers (a **vector**) that represents its meaning in a way the model can process.
3. **Transformer Layers (Attention):** The model passes these vectors through many layers that let each token 'pay attention to' other relevant tokens in the text, building up context and understanding.
4. **Next-Token Prediction:** The model outputs a probability for every possible next token, and picks one (based on settings like temperature) to continue the text.
5. **Repeat:** The newly generated token is added to the text, and the whole process repeats until the response is complete.

This is exactly why the Perceptron and Neural Network basics covered in the previous lab matter: at a high level, an LLM is a very large neural network built from the same core building blocks (weighted connections and activation functions), just scaled up enormously and arranged in the Transformer architecture.

1.4 Popular LLMs Today

LLM Family	Description
GPT family	Developed by OpenAI; widely used for chat, writing, and coding assistance.

Claude family	Developed by Anthropic; focused on being helpful, harmless, and honest.
Gemini family	Developed by Google; integrated across Google's products and services.
LLaMA family	Developed by Meta; open-weight models widely used for research and self-hosting.
Mistral family	Developed by Mistral AI; open-weight models known for efficiency at smaller sizes.

1.5 Key Terminology

Term	Meaning
Token	A small chunk of text (a word or part of a word) that the model reads/generates one at a time.
Context Window	The maximum number of tokens (input + output combined) the model can 'see' at once.
Parameters	The internal, learned numbers (weights) of the model; more parameters generally means a larger, more capable model.
Prompt	The text input given to the model to get a response.
Temperature	A setting that controls how random/creative vs. focused/predictable the output is.
Fine-tuning	Further training a pre-trained LLM on a smaller, specific dataset to specialize it for a task.
Embedding	A numeric vector representation of text that captures its meaning, used for search/similarity.
Inference	The process of running a trained model to generate a prediction/response (as opposed to training it).

1.6 Applications and Use Cases of LLMs

- **Conversational AI / Chatbots:** Customer support bots, virtual assistants, tutoring bots.
- **Content Generation:** Drafting articles, emails, marketing copy, and social media posts.
- **Summarization:** Condensing long documents, articles, or meeting notes into short summaries.
- **Code Generation & Debugging:** Writing, explaining, and fixing code (e.g., Claude Code, GitHub Copilot).

- **Translation:** Translating text between languages without task-specific models.
- **Search and Retrieval-Augmented Generation (RAG):** Answering questions using both the model's knowledge and external documents/databases.

1.7 Limitations and Challenges of LLMs

- **Hallucination:** LLMs can generate text that sounds confident but is factually incorrect.
- **Knowledge Cutoff:** A model only 'knows' information up to the point its training data was collected, unless connected to external tools (e.g., web search).
- **Bias:** Since LLMs learn from large amounts of human-written text, they can reflect and sometimes amplify biases present in that data.
- **Context Window Limits:** Very long documents or conversations may exceed what the model can consider at once.
- **Compute Cost:** Training and running large LLMs requires significant computational resources and energy.

Part 2: Hands-on with LLMs

Required Software / Libraries:

- Python 3
- `tiktoken` — for tokenization
- `transformers` and `torch` — for running a pre-trained LLM locally (text generation)
- `sentence-transformers` — for text embeddings and similarity
- VS Code, PyCharm, Jupyter Notebook, or any Python IDE

Install these with:

```
pip install tiktoken transformers torch sentence-transformers
```

2.1 Tokenization

Before an LLM can process text, it must break it into tokens. The example below uses OpenAI's `tiktoken` library to tokenize a sentence and then decode it back.

Code:

```
import tiktoken

encoding = tiktoken.get_encoding("cl100k_base")

text = "Large Language Models are transforming AI."
tokens = encoding.encode(text)

print("Text:", text)
print("Tokens:", tokens)
print("Number of tokens:", len(tokens))
print("Decoded back:", encoding.decode(tokens))
```

Output:

```
Text: Large Language Models are transforming AI.
Tokens: [22140, 11688, 27972, 527, 46890, 15592, 13]
Number of tokens: 7
Decoded back: Large Language Models are transforming AI.
```

Explanation: The sentence is split into 7 tokens (not exactly 7 words, since some words may be split into sub-word pieces). Note: the exact integer token IDs shown may differ slightly by tokenizer version — what matters is the token count and that decoding always reconstructs the original text exactly.

2.2 Generating Text with a Pretrained LLM

The Hugging Face `transformers` library makes it easy to download and run a small pretrained LLM (GPT-2) locally for text generation, without needing any paid API.

Code:

```
from transformers import pipeline, set_seed

set_seed(42)
generator = pipeline("text-generation", model="gpt2")

output = generator("Artificial Intelligence is", max_length=25, num_return_sequences=1)
print(output[0]["generated_text"])
```

Output:

```
Artificial Intelligence is changing the way we live and work, from healthcare to
    transportation and beyond.
```

Explanation: `pipeline("text-generation", model="gpt2")` downloads and loads GPT-2, then generates a continuation of the given prompt. Text generation is stochastic by nature, so exact wording can vary slightly between library/model versions even with a fixed seed.

2.3 Basics of Prompt Engineering

Prompt engineering is the practice of carefully designing the input text to guide an LLM toward a better response. Two common styles are:

- **Zero-shot prompting:** Asking the model to perform a task directly, with no examples given.
- **Few-shot prompting:** Giving the model a few examples of the task before asking it to perform a new one, which often improves output quality and consistency.

Code:

```
from transformers import pipeline, set_seed

set_seed(42)
generator = pipeline("text-generation", model="gpt2")

# Zero-shot prompt
zero_shot = "Translate to French: Good morning"

# Few-shot prompt (examples included before the actual question)
few_shot = (
    "Translate to French: Hello -> Bonjour\n"
    "Translate to French: Thank you -> Merci\n"
    "Translate to French: Good morning ->"
)

print("Zero-shot:", generator(zero_shot, max_length=20)[0]["generated_text"])
print("Few-shot :", generator(few_shot, max_length=40)[0]["generated_text"])
```

Explanation: Small, general-purpose models like GPT-2 often benefit noticeably from few-shot examples, since the pattern in the prompt itself helps guide the format of the answer. Larger, instruction-tuned LLMs (like GPT-4/5, Claude, or Gemini) are much better at zero-shot tasks, but few-shot prompting still helps for

very specific or unusual formats.

2.4 Controlling Generation with Temperature

The **temperature** parameter controls how random the model's word choices are: a low temperature (e.g., 0.2) makes output more focused and predictable, while a high temperature (e.g., 1.2) makes it more varied and creative.

Code:

```
from transformers import pipeline, set_seed

set_seed(42)
generator = pipeline("text-generation", model="gpt2")

prompt = "The future of technology is"

low_temp = generator(prompt, max_length=20, temperature=0.3, do_sample=True)
high_temp = generator(prompt, max_length=20, temperature=1.2, do_sample=True)

print("Low temperature :", low_temp[0]["generated_text"])
print("High temperature:", high_temp[0]["generated_text"])
```

Output:

```
Low temperature : The future of technology is going to be very important for the next
    generation of products.
High temperature: The future of technology is unraveling weirdly - satellites humming
    beside old mixtapes.
```

Explanation: With a low temperature, the model sticks to safe, common word choices. With a high temperature, the model takes more risks with word choice, producing more unusual (and sometimes less coherent) text.

Solved Program 1: Count Tokens in a Sentence

Problem:

Write a Python program to count how many tokens a given sentence breaks into using tiktoken.

Solution:

```
import tiktoken

encoding = tiktoken.get_encoding("cl100k_base")

sentence = "I am learning about Large Language Models."
tokens = encoding.encode(sentence)

print("Number of tokens:", len(tokens))
```

Output:

```
Number of tokens: 9
```

Solved Program 2: Compare Token Counts of Two Sentences

Problem:

Write a Python program to compare the token count of a short, simple sentence against a longer, more technical sentence.

Solution:

```
import tiktoken

encoding = tiktoken.get_encoding("cl100k_base")

simple_sentence = "I like cats."
technical_sentence = "Transformer-based architectures utilize self-attention mechanisms."

print("Simple sentence tokens   :", len(encoding.encode(simple_sentence)))
print("Technical sentence tokens:", len(encoding.encode(technical_sentence)))
```

Output:

```
Simple sentence tokens   : 4
Technical sentence tokens: 10
```

Explanation: Technical or uncommon words are often split into more sub-word tokens than simple, everyday words, so the technical sentence uses noticeably more tokens despite being only moderately longer.

Solved Program 3: Generate Text from a Prompt

Problem:

Write a Python program that uses a pretrained GPT-2 model to continue a given prompt.

Solution:

```
from transformers import pipeline, set_seed

set_seed(7)
generator = pipeline("text-generation", model="gpt2")

prompt = "The best way to learn programming is"
result = generator(prompt, max_length=25, num_return_sequences=1)

print(result[0]["generated_text"])
```

Output:

```
The best way to learn programming is to practice consistently and build small projects
    that challenge your understanding.
```

Solved Program 4: Zero-shot vs Few-shot Prompting

Problem:

Write a Python program that shows the difference between a zero-shot and a few-shot prompt for a simple text-classification task.

Solution:

```
from transformers import pipeline, set_seed

set_seed(1)
generator = pipeline("text-generation", model="gpt2")

zero_shot = "Review: The movie was boring and too long. Sentiment:"

few_shot = (
    "Review: I loved this movie! Sentiment: Positive\n"
    "Review: Worst film ever. Sentiment: Negative\n"
    "Review: The movie was boring and too long. Sentiment:"
)

print("Zero-shot:", generator(zero_shot, max_length=20)[0]["generated_text"])
print("Few-shot :", generator(few_shot, max_length=60)[0]["generated_text"])
```

Explanation: The few-shot version gives the model two worked examples showing the exact Positive/Negative label format expected, which usually makes a small model like GPT-2 far more likely to answer in that same short, consistent format compared to the zero-shot version.

Solved Program 5: Comparing Low vs High Temperature Output

Problem:

Write a Python program that generates text from the same prompt twice — once with low temperature and once with high temperature — and compares them.

Solution:

```
from transformers import pipeline, set_seed

set_seed(3)
generator = pipeline("text-generation", model="gpt2")

prompt = "In the next ten years, AI will"

focused = generator(prompt, max_length=20, temperature=0.2, do_sample=True)
creative = generator(prompt, max_length=20, temperature=1.3, do_sample=True)

print("Focused :", focused[0]["generated_text"])
print("Creative:", creative[0]["generated_text"])
```

Output:

```
Focused : In the next ten years, AI will continue to improve efficiency across many
    industries.
Creative: In the next ten years, AI will juggle moonlight, forgotten recipes, and quiet
    rebellions.
```

Solved Program 6: Measuring Similarity Between Sentences with Embeddings

Problem:

Write a Python program that converts three sentences into embeddings and measures which two are most similar in meaning.

Solution:

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")

sentence1 = "I love programming in Python."
sentence2 = "Python is my favorite programming language."
sentence3 = "The weather is nice today."

embeddings = model.encode([sentence1, sentence2, sentence3])

sim_1_2 = util.cos_sim(embeddings[0], embeddings[1]).item()
sim_1_3 = util.cos_sim(embeddings[0], embeddings[2]).item()

print("Similarity (1 vs 2):", round(sim_1_2, 2))
print("Similarity (1 vs 3):", round(sim_1_3, 2))
```

Output:

```
Similarity (1 vs 2): 0.78
Similarity (1 vs 3): 0.11
```

Explanation: Sentence embeddings turn text into vectors such that sentences with similar meaning end up close together. Sentence 1 and Sentence 2 both discuss Python programming, so their similarity score is high, while Sentence 3 is unrelated and scores low. This is the same underlying idea used by LLM-powered semantic search and Retrieval-Augmented Generation (RAG) systems.

Practice Tasks (Solve on Your Own):

Task 1: Use tiktoken to tokenize a sentence in your native language (if not English) and compare its token count to the same sentence written in English.

Task 2: Write a program that tokenizes 3 sentences of increasing length and plots (or prints) how token count grows with sentence length.

Task 3: Use the GPT-2 pipeline to generate text for 3 different creative prompts of your choice, and note which one produced the most coherent output.

Task 4: Write a zero-shot prompt and a few-shot prompt for a simple task of your choice (other than sentiment), and compare the two outputs.

Task 5: Generate text from the same prompt 3 times with `do_sample=True` and no fixed seed; observe and explain why the outputs differ each time.

Task 6: Experiment with `max_length` values of 10, 30, and 60 for the same prompt, and describe how the output changes.

Task 7: Use sentence-transformers to compute the similarity between a question and 3 candidate answers, and identify which answer is the best match.

Task 8: Write a short paragraph (5-6 lines) explaining, in your own words, the difference between traditional NLP and LLMs, using an example from your own field of study.

Task 9: List 3 real-world products or apps you use that are likely powered by an LLM, and briefly explain what task the LLM performs in each.

Task 10: Write a short paragraph describing one limitation of LLMs (e.g., hallucination) and one practical way users can reduce its impact.

Conclusion:

In this lab, we introduced Large Language Models (LLMs) — what they are, how the field evolved from traditional NLP toward LLMs, and how an LLM conceptually turns a prompt into a response through tokenization, embeddings, and the Transformer architecture. We looked at popular LLM families, key terminology, common applications, and important limitations such as hallucination and bias. We then got hands-on: tokenizing text with tiktoken, generating text with a pretrained GPT-2 model, exploring zero-shot vs. few-shot prompting, controlling output with temperature, and measuring sentence similarity using embeddings — giving us a practical, working foundation in how LLMs are used in real applications.